Q - 1

In [2]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

In [ ]:
```
import gensim.downloader as api
from sklearn.decomposition import PCA
from nltk.tokenize import word_tokenize
```

In [ ]:
```
import pandas as pd
import numpy as np
import nltk
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, Dataset
import matplotlib.pyplot as plt
```

In [9]:
```
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Out[9]:
```
True
```

In [5]:
```
dataset=pd.read_csv("/content/drive/MyDrive/MLSP3/IMDB Dataset.csv")
```

In [6]:
```
train_set, temp_set = train_test_split(dataset, test_size=0.2, random_state=42)
test_set, val_set = train_test_split(temp_set, test_size=0.5, random_state=42)
```

In [7]:
```
word2vec_model = api.load("word2vec-google-news-300")
```

```
[==================================================] 100.0% 1662.8/1662.8MB download
ed
```

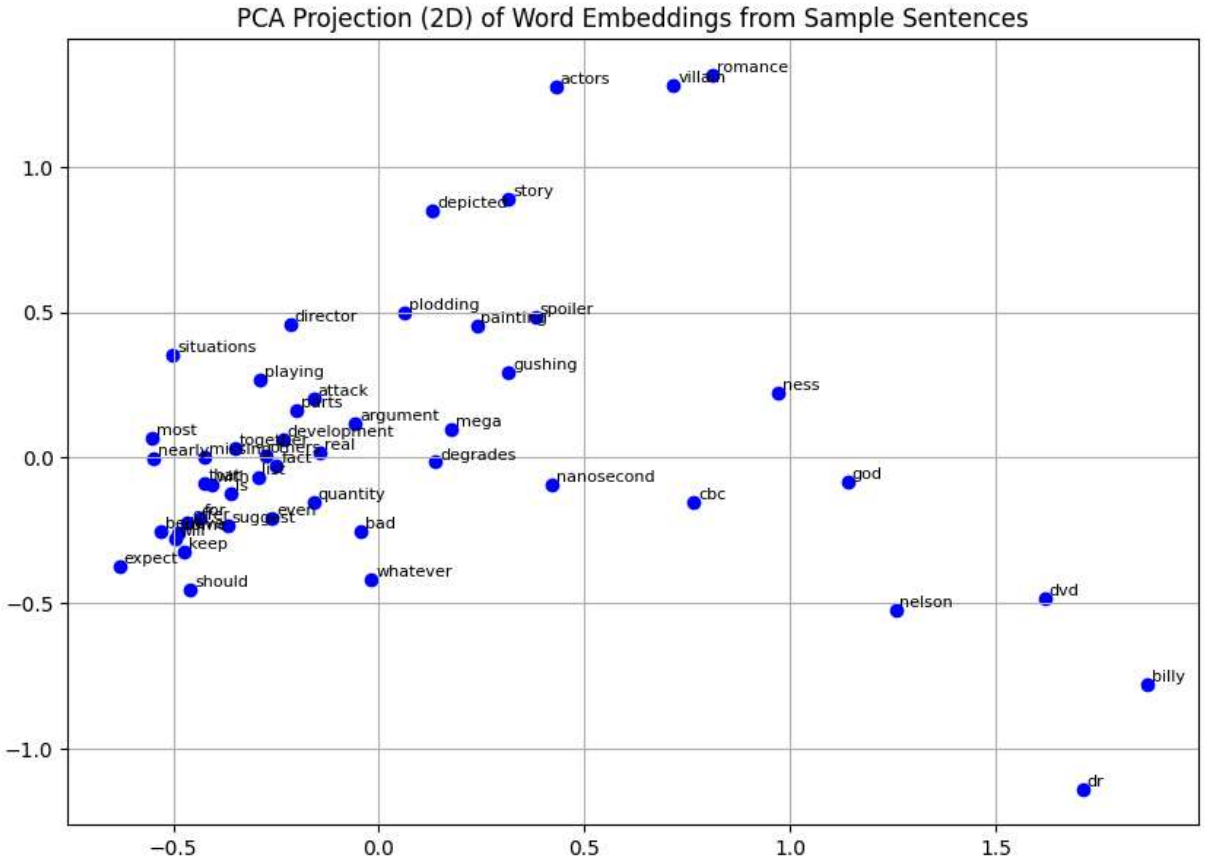
PART A

In [ ]:
```
text_column = "review"
label_column = "sentiment"
validation_texts = list(val_set[text_column])
example_sentences = validation_texts[:10]
processed_sentences = []
```

In [ ]:
```
for sentence in example_sentences:
    words = word_tokenize(sentence.lower())
    known_words = [word for word in words if word in word2vec_model]
    processed_sentences.append(known_words)
```

```
collected_words = [word for group in processed_sentences for word in group]
distinct_words = list(set(collected_words))
print(f"Number of distinct words: {len(distinct_words)}")

embedding_vectors = np.array([word2vec_model[word] for word in distinct_words])
pca_model = PCA(n_components=2)
reduced_embeddings = pca_model.fit_transform(embedding_vectors)

plt.figure(figsize=(10, 7))
for idx, word in enumerate(distinct_words[:50]):
    x, y = reduced_embeddings[idx]
    plt.scatter(x, y, color='blue')
    plt.text(x + 0.01, y + 0.01, word, fontsize=8)

plt.title("PCA Projection (2D) of Word Embeddings from Sample Sentences")
plt.grid(True)
plt.show()
```

```
Number of distinct words: 647
```



PART B

In [27]:
```
def preprocess_and_embed(data, word2vec_model):
    reviews = []
    labels = []
    for _, row in data.iterrows():
        words = nltk.word_tokenize(row['review'].lower())
```

```
        embeddings = [word2vec_model[word] for word in words if word in word2vec_mo
        if embeddings:
            reviews.append(np.mean(embeddings, axis=0))
        else:
            reviews.append(np.zeros(300))
        labels.append(1 if row['sentiment'] == 'positive' else 0)
    return np.array(reviews), np.array(labels)
```

In [25]:
```
train_data, test_data = train_test_split(dataset, test_size=0.1, random_state=42)
validation_data, test_data = train_test_split(test_data, test_size=0.5, random_stat
```

In [26]:
```
X_train, y_train = preprocess_and_embed(train_data, word2vec_model)
X_val, y_val = preprocess_and_embed(validation_data, word2vec_model)
X_test, y_test = preprocess_and_embed(test_data, word2vec_model)
```

In [28]:
```
class SentimentDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

In [29]:
```
batch_size = 32
train_loader = DataLoader(SentimentDataset(X_train, y_train), batch_size=batch_size
val_loader = DataLoader(SentimentDataset(X_val, y_val), batch_size=batch_size)
test_loader = DataLoader(SentimentDataset(X_test, y_test), batch_size=batch_size)
```

In [ ]:
```
class SentimentLSTM(nn.Module):
    def __init__(self):
        super(SentimentLSTM, self).__init__()
        self.lstm = nn.LSTM(input_size=300, hidden_size=256, num_layers=2, batch_fi
        self.fc = nn.Linear(256, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        out, _ = self.lstm(x.unsqueeze(1))
        out = out.mean(dim=1)
        out = self.fc(out)
        return self.sigmoid(out)
```

In [31]:
```
model = SentimentLSTM()
```

In [32]:
```
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.BCELoss()
```

In [33]:
```
epochs = 25
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

```
train_losses = []
val_losses = []
```

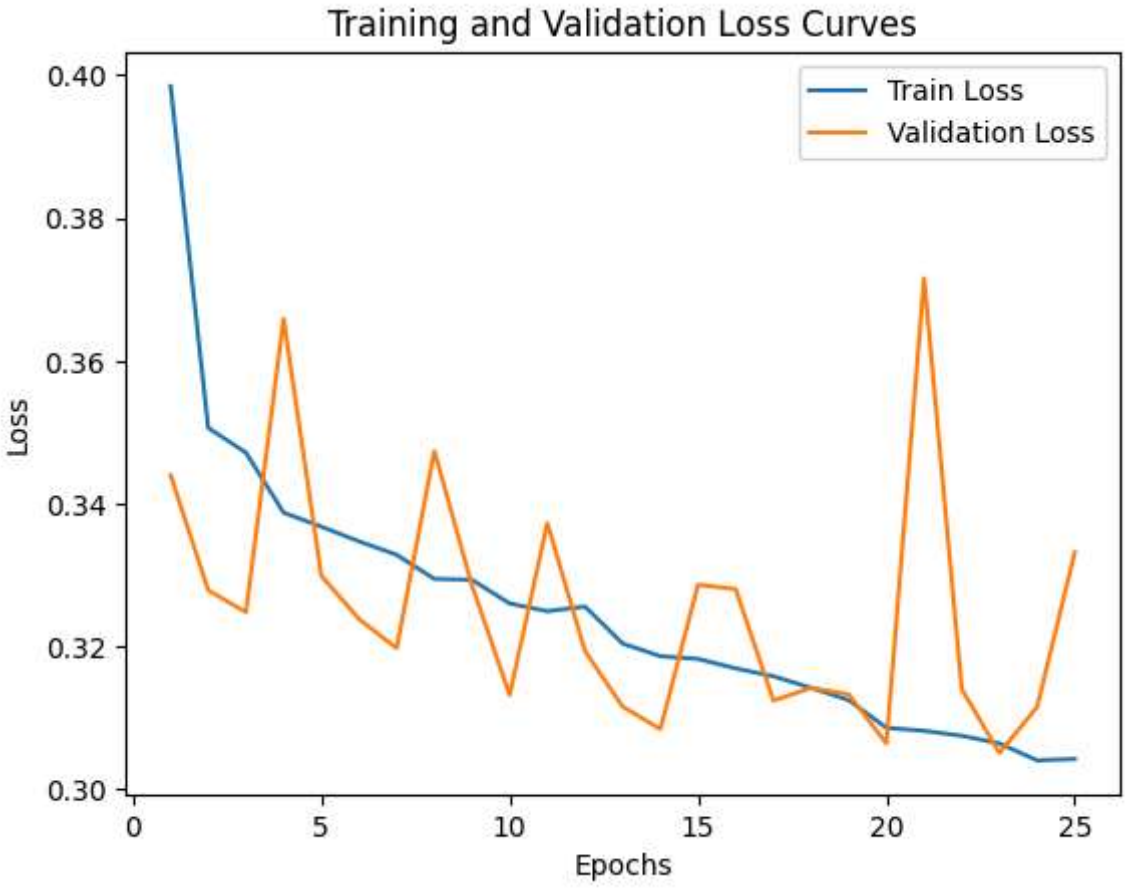
In [34]:

```
for epoch in range(epochs):
    model.train()
    train_loss = 0.0

    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer.zero_grad()
        outputs = model(X_batch).squeeze()
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()

    train_loss /= len(train_loader)
    train_losses.append(train_loss)

    # Validation loop
    model.eval()
    val_loss = 0.0

    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)

            outputs = model(X_batch).squeeze()
            loss = criterion(outputs, y_batch)
            val_loss += loss.item()

    val_loss /= len(val_loader)
    val_losses.append(val_loss)

    print(f"Epoch {epoch + 1}/{epochs}, Train Loss: {train_loss:.4f}, Validation Lo
```

```
Epoch 1/25, Train Loss: 0.3984, Validation Loss: 0.3440
Epoch 2/25, Train Loss: 0.3506, Validation Loss: 0.3279
Epoch 3/25, Train Loss: 0.3471, Validation Loss: 0.3248
Epoch 4/25, Train Loss: 0.3388, Validation Loss: 0.3659
Epoch 5/25, Train Loss: 0.3368, Validation Loss: 0.3299
Epoch 6/25, Train Loss: 0.3347, Validation Loss: 0.3239
Epoch 7/25, Train Loss: 0.3329, Validation Loss: 0.3198
Epoch 8/25, Train Loss: 0.3295, Validation Loss: 0.3474
Epoch 9/25, Train Loss: 0.3294, Validation Loss: 0.3286
Epoch 10/25, Train Loss: 0.3260, Validation Loss: 0.3132
Epoch 11/25, Train Loss: 0.3250, Validation Loss: 0.3373
Epoch 12/25, Train Loss: 0.3256, Validation Loss: 0.3194
Epoch 13/25, Train Loss: 0.3204, Validation Loss: 0.3116
Epoch 14/25, Train Loss: 0.3187, Validation Loss: 0.3085
Epoch 15/25, Train Loss: 0.3183, Validation Loss: 0.3287
Epoch 16/25, Train Loss: 0.3170, Validation Loss: 0.3280
Epoch 17/25, Train Loss: 0.3158, Validation Loss: 0.3124
Epoch 18/25, Train Loss: 0.3142, Validation Loss: 0.3142
Epoch 19/25, Train Loss: 0.3125, Validation Loss: 0.3133
Epoch 20/25, Train Loss: 0.3086, Validation Loss: 0.3065
Epoch 21/25, Train Loss: 0.3082, Validation Loss: 0.3715
Epoch 22/25, Train Loss: 0.3075, Validation Loss: 0.3140
Epoch 23/25, Train Loss: 0.3065, Validation Loss: 0.3051
Epoch 24/25, Train Loss: 0.3041, Validation Loss: 0.3116
Epoch 25/25, Train Loss: 0.3043, Validation Loss: 0.3332
```

In [35]:

```
plt.plot(range(1, epochs + 1), train_losses, label="Train Loss")
plt.plot(range(1, epochs + 1), val_losses, label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training and Validation Loss Curves")
plt.legend()
plt.show()
```



In [36]:
```python
# Evaluate on the test set
model.eval()
correct_predictions = 0

with torch.no_grad():
    for X_batch, y_batch in test_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        outputs = model(X_batch).squeeze()
        predictions = (outputs >= 0.5).float()
        correct_predictions += (predictions == y_batch).sum().item()

test_accuracy = correct_predictions / len(test_loader.dataset)
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
Test Accuracy: 0.8508
```

PART C

In [37]:
```python
class AttentionLayer(nn.Module):
    def __init__(self, hidden_dim):
        super(AttentionLayer, self).__init__()
        self.attention_weights = nn.Linear(hidden_dim, 1)

    def forward(self, lstm_output):
        weights = torch.softmax(self.attention_weights(lstm_output), dim=1)
```

```
        context_vector = torch.sum(weights * lstm_output, dim=1)  # Weighted sum of
        return context_vector
```

In [38]:
```
class SentimentLSTMWithAttention(nn.Module):
    def __init__(self):
        super(SentimentLSTMWithAttention, self).__init__()
        self.lstm = nn.LSTM(input_size=300, hidden_size=256, num_layers=2, batch_fi
        self.attention = AttentionLayer(hidden_dim=256)
        self.fc = nn.Linear(256, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        lstm_output, _ = self.lstm(x.unsqueeze(1))  # Add sequence dimension
        context_vector = self.attention(lstm_output)
        out = self.fc(context_vector)
        return self.sigmoid(out)
```

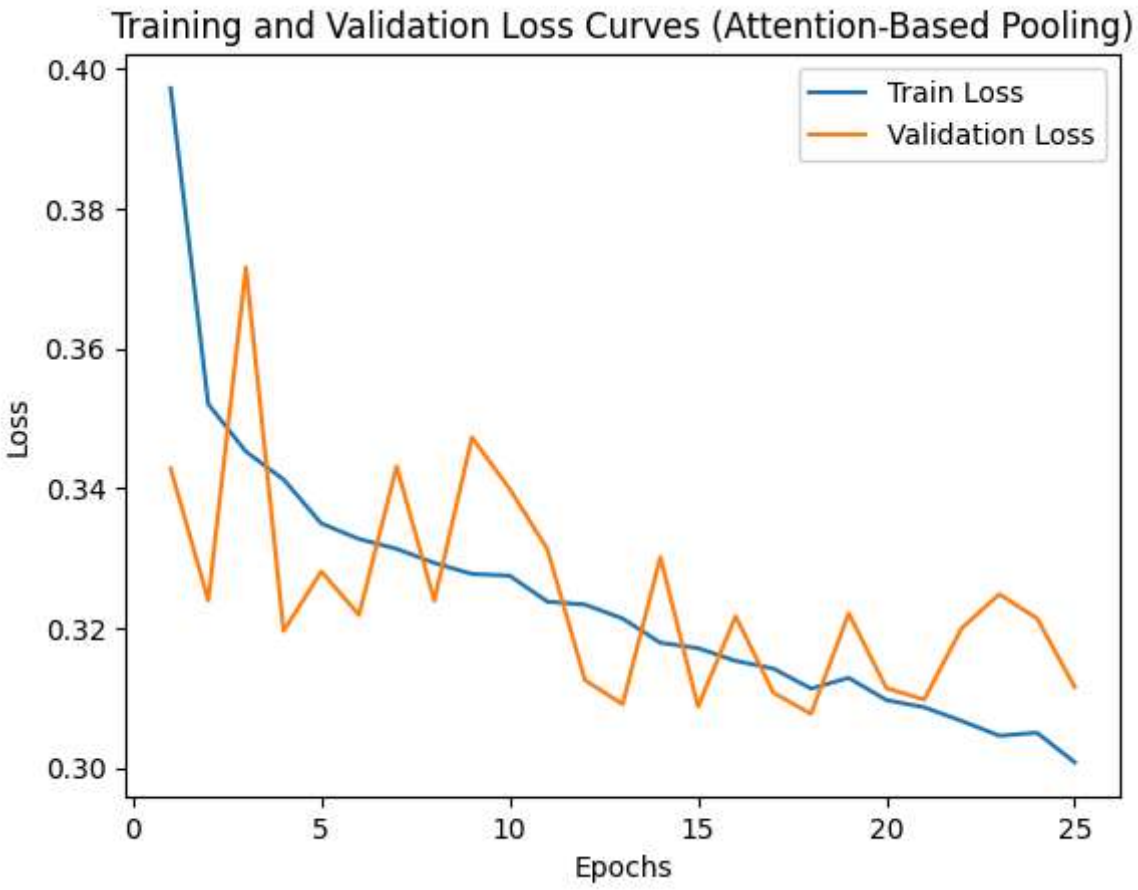
In [39]:
```
model_with_attention = SentimentLSTMWithAttention()
optimizer_attention = optim.Adam(model_with_attention.parameters(), lr=1e-3)
criterion_attention = nn.BCELoss()
```

In [40]:
```
epochs = 25
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model_with_attention.to(device)

train_losses = []
val_losses = []

for epoch in range(epochs):
    model_with_attention.train()
    train_loss = 0.0

    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer_attention.zero_grad()
        outputs = model_with_attention(X_batch).squeeze()
        loss = criterion_attention(outputs, y_batch)
        loss.backward()
        optimizer_attention.step()

        train_loss += loss.item()

    train_loss /= len(train_loader)
    train_losses.append(train_loss)

    # Validation loop
    model_with_attention.eval()
    val_loss = 0.0

    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)

            outputs = model_with_attention(X_batch).squeeze()
```

```
            loss = criterion_attention(outputs, y_batch)
            val_loss += loss.item()

    val_loss /= len(val_loader)
    val_losses.append(val_loss)

    print(f"Epoch {epoch + 1}/{epochs}, Train Loss: {train_loss:.4f}, Validation Lo
```

```
Epoch 1/25, Train Loss: 0.3972, Validation Loss: 0.3428
Epoch 2/25, Train Loss: 0.3520, Validation Loss: 0.3239
Epoch 3/25, Train Loss: 0.3452, Validation Loss: 0.3716
Epoch 4/25, Train Loss: 0.3412, Validation Loss: 0.3196
Epoch 5/25, Train Loss: 0.3350, Validation Loss: 0.3281
Epoch 6/25, Train Loss: 0.3327, Validation Loss: 0.3219
Epoch 7/25, Train Loss: 0.3313, Validation Loss: 0.3431
Epoch 8/25, Train Loss: 0.3293, Validation Loss: 0.3239
Epoch 9/25, Train Loss: 0.3277, Validation Loss: 0.3473
Epoch 10/25, Train Loss: 0.3275, Validation Loss: 0.3399
Epoch 11/25, Train Loss: 0.3238, Validation Loss: 0.3313
Epoch 12/25, Train Loss: 0.3234, Validation Loss: 0.3126
Epoch 13/25, Train Loss: 0.3214, Validation Loss: 0.3091
Epoch 14/25, Train Loss: 0.3179, Validation Loss: 0.3301
Epoch 15/25, Train Loss: 0.3171, Validation Loss: 0.3088
Epoch 16/25, Train Loss: 0.3153, Validation Loss: 0.3217
Epoch 17/25, Train Loss: 0.3142, Validation Loss: 0.3108
Epoch 18/25, Train Loss: 0.3114, Validation Loss: 0.3077
Epoch 19/25, Train Loss: 0.3129, Validation Loss: 0.3221
Epoch 20/25, Train Loss: 0.3097, Validation Loss: 0.3114
Epoch 21/25, Train Loss: 0.3087, Validation Loss: 0.3098
Epoch 22/25, Train Loss: 0.3067, Validation Loss: 0.3200
Epoch 23/25, Train Loss: 0.3046, Validation Loss: 0.3248
Epoch 24/25, Train Loss: 0.3050, Validation Loss: 0.3214
Epoch 25/25, Train Loss: 0.3008, Validation Loss: 0.3116
```

In [41]:
```
plt.plot(range(1, epochs + 1), train_losses, label="Train Loss")
plt.plot(range(1, epochs + 1), val_losses, label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training and Validation Loss Curves (Attention-Based Pooling)")
plt.legend()
plt.show()
```



In [42]:
```
model_with_attention.eval()
correct_predictions = 0

with torch.no_grad():
    for X_batch, y_batch in test_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        outputs = model_with_attention(X_batch).squeeze()
        predictions = (outputs >= 0.5).float()
        correct_predictions += (predictions == y_batch).sum().item()

test_accuracy = correct_predictions / len(test_loader.dataset)
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
Test Accuracy: 0.8548
```

PART D

In [43]:
```
embedding_dim = 300
train_data, test_data = train_test_split(dataset, test_size=0.1, random_state=42)
validation_data, test_data = train_test_split(test_data, test_size=0.5, random_stat

X_train, y_train = preprocess_and_embed(train_data, word2vec_model)
X_val, y_val = preprocess_and_embed(validation_data, word2vec_model)
X_test, y_test = preprocess_and_embed(test_data, word2vec_model)
```

In [44]:
```
# Dataset class
class SentimentDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

In [45]:
```
batch_size = 32
train_loader = DataLoader(SentimentDataset(X_train, y_train), batch_size=batch_size
val_loader = DataLoader(SentimentDataset(X_val, y_val), batch_size=batch_size)
test_loader = DataLoader(SentimentDataset(X_test, y_test), batch_size=batch_size)
embedding_matrix = np.random.normal(0, 1, (1, embedding_dim))
```

In [46]:
```
# Transformer Model
class TransformerModel(nn.Module):
    def __init__(self, embedding_matrix, hidden_dim=256, num_heads=4, dropout=0.3):
        super(TransformerModel, self).__init__()
        _, embedding_dim = embedding_matrix.shape
        self.linear = nn.Linear(embedding_dim, embedding_dim)
        self.pos_embedding = nn.Parameter(torch.randn(1, 1, embedding_dim))

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embedding_dim,
            nhead=num_heads,
            dim_feedforward=hidden_dim,
            dropout=dropout
        )
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=

        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(embedding_dim, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.linear(x).unsqueeze(1) + self.pos_embedding
        x = x.permute(1, 0, 2)
        x = self.transformer_encoder(x)
        x = x.permute(1, 0, 2).squeeze(1)
        x = self.dropout(x)
        return self.sigmoid(self.fc(x))
```

In [ ]:
```
transformer_model = TransformerModel(embedding_matrix).to(device)
optimizer_transformer = optim.Adam(transformer_model.parameters(), lr=1e-3)
criterion_transformer = nn.BCELoss()
```

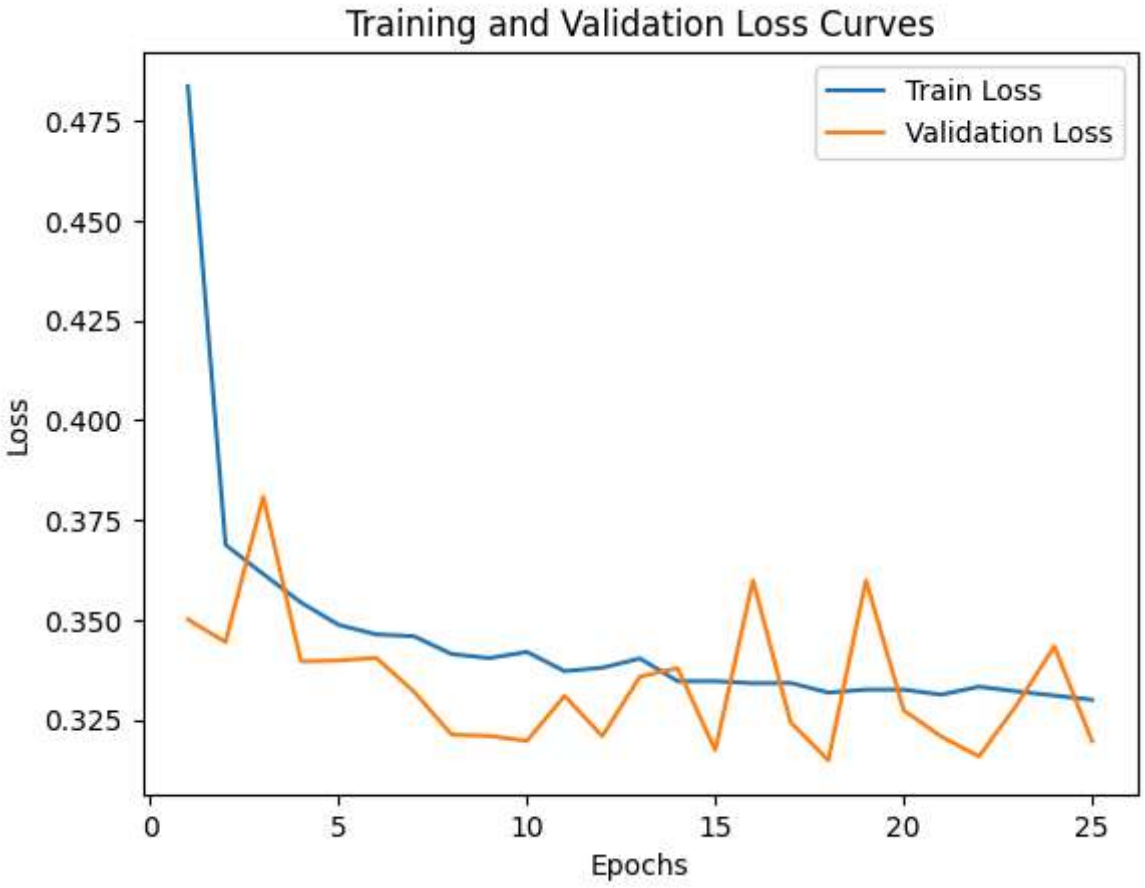
In [48]:
```
# Training loop
epochs = 25
train_losses = []
val_losses = []
```

```
for epoch in range(epochs):
    transformer_model.train()
    train_loss = 0.0

    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer_transformer.zero_grad()
        outputs = transformer_model(X_batch).squeeze()
        loss = criterion_transformer(outputs, y_batch)
        loss.backward()
        optimizer_transformer.step()
        train_loss += loss.item()

    train_loss /= len(train_loader)
    train_losses.append(train_loss)

    # Validation loop
    transformer_model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            outputs = transformer_model(X_batch).squeeze()
            loss = criterion_transformer(outputs, y_batch)
            val_loss += loss.item()

    val_loss /= len(val_loader)
    val_losses.append(val_loss)

    print(f"Epoch {epoch + 1}/{epochs}, Train Loss: {train_loss:.4f}, Validation Lo
```

```
Epoch 1/25, Train Loss: 0.4836, Validation Loss: 0.3501
Epoch 2/25, Train Loss: 0.3688, Validation Loss: 0.3444
Epoch 3/25, Train Loss: 0.3615, Validation Loss: 0.3808
Epoch 4/25, Train Loss: 0.3544, Validation Loss: 0.3396
Epoch 5/25, Train Loss: 0.3488, Validation Loss: 0.3398
Epoch 6/25, Train Loss: 0.3463, Validation Loss: 0.3405
Epoch 7/25, Train Loss: 0.3459, Validation Loss: 0.3320
Epoch 8/25, Train Loss: 0.3415, Validation Loss: 0.3213
Epoch 9/25, Train Loss: 0.3404, Validation Loss: 0.3209
Epoch 10/25, Train Loss: 0.3420, Validation Loss: 0.3197
Epoch 11/25, Train Loss: 0.3372, Validation Loss: 0.3310
Epoch 12/25, Train Loss: 0.3380, Validation Loss: 0.3209
Epoch 13/25, Train Loss: 0.3403, Validation Loss: 0.3358
Epoch 14/25, Train Loss: 0.3347, Validation Loss: 0.3379
Epoch 15/25, Train Loss: 0.3347, Validation Loss: 0.3174
Epoch 16/25, Train Loss: 0.3341, Validation Loss: 0.3599
Epoch 17/25, Train Loss: 0.3342, Validation Loss: 0.3243
Epoch 18/25, Train Loss: 0.3318, Validation Loss: 0.3148
Epoch 19/25, Train Loss: 0.3325, Validation Loss: 0.3599
Epoch 20/25, Train Loss: 0.3325, Validation Loss: 0.3274
Epoch 21/25, Train Loss: 0.3313, Validation Loss: 0.3207
Epoch 22/25, Train Loss: 0.3333, Validation Loss: 0.3158
Epoch 23/25, Train Loss: 0.3321, Validation Loss: 0.3287
Epoch 24/25, Train Loss: 0.3310, Validation Loss: 0.3434
Epoch 25/25, Train Loss: 0.3300, Validation Loss: 0.3196
```

In [49]:

```python
# Plot losses
plt.plot(range(1, epochs + 1), train_losses, label="Train Loss")
plt.plot(range(1, epochs + 1), val_losses, label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training and Validation Loss Curves")
plt.legend()
plt.show()
```



In [50]:
```python
# Evaluate
transformer_model.eval()
correct_predictions = 0
with torch.no_grad():
    for X_batch, y_batch in test_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        outputs = transformer_model(X_batch).squeeze()
        predictions = (outputs >= 0.5).float()
        correct_predictions += (predictions == y_batch).sum().item()

test_accuracy = correct_predictions / len(test_loader.dataset)
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
Test Accuracy: 0.8556
```

Conclusion

In this question, we explored various deep learning architectures for sentiment analysis on the IMDB dataset. We started with a basic LSTM model, enhanced it with an attention mechanism, and finally implemented a Transformer-based model. Below are the key takeaways:

1. **LSTM Model**: The LSTM model provided a solid baseline for sentiment analysis, effectively capturing sequential dependencies in the text data.

2. **Attention Mechanism**: Adding an attention mechanism improved the model's performance by allowing it to focus on the most relevant parts of the input sequence, leading to better interpretability and accuracy.

3. **Transformer Model**: The Transformer-based model demonstrated the power of self-attention mechanisms, achieving competitive results and showcasing its ability to handle long-range dependencies in text.

4. **Performance Comparison**: Each model was evaluated on training, validation, and test sets. The Transformer model showed promising results, but the choice of the best model depends on the specific use case, computational resources, and desired trade-offs between accuracy and efficiency.

This question highlights the importance of experimenting with different architectures and techniques to achieve optimal performance in natural language processing tasks. Future work could involve fine-tuning pre-trained models like BERT or GPT for further improvements.

In [1]:
```
!pip install pandas librosa
```

```
Requirement already satisfied: pandas in /data/home/kaushikk/anaconda3/lib/python3.1
2/site-packages (2.2.2)
Requirement already satisfied: librosa in /data/home/kaushikk/anaconda3/lib/python3.
12/site-packages (0.11.0)
Requirement already satisfied: numpy>=1.26.0 in /data/home/kaushikk/anaconda3/lib/py
thon3.12/site-packages (from pandas) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /data/home/kaushikk/anacond
a3/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /data/home/kaushikk/anaconda3/lib/pyt
hon3.12/site-packages (from pandas) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /data/home/kaushikk/anaconda3/lib/p
ython3.12/site-packages (from pandas) (2023.3)
Requirement already satisfied: audioread>=2.1.9 in /data/home/kaushikk/anaconda3/li
b/python3.12/site-packages (from librosa) (3.0.1)
Requirement already satisfied: numba>=0.51.0 in /data/home/kaushikk/anaconda3/lib/py
thon3.12/site-packages (from librosa) (0.60.0)
Requirement already satisfied: scipy>=1.6.0 in /data/home/kaushikk/anaconda3/lib/pyt
hon3.12/site-packages (from librosa) (1.13.1)
Requirement already satisfied: scikit-learn>=1.1.0 in /data/home/kaushikk/anaconda3/
lib/python3.12/site-packages (from librosa) (1.5.1)
Requirement already satisfied: joblib>=1.0 in /data/home/kaushikk/anaconda3/lib/pyth
on3.12/site-packages (from librosa) (1.4.2)
Requirement already satisfied: decorator>=4.3.0 in /data/home/kaushikk/anaconda3/li
b/python3.12/site-packages (from librosa) (5.1.1)
Requirement already satisfied: soundfile>=0.12.1 in /data/home/kaushikk/anaconda3/li
b/python3.12/site-packages (from librosa) (0.13.1)
Requirement already satisfied: pooch>=1.1 in /data/home/kaushikk/anaconda3/lib/pytho
n3.12/site-packages (from librosa) (1.8.2)
Requirement already satisfied: soxr>=0.3.2 in /data/home/kaushikk/anaconda3/lib/pyth
on3.12/site-packages (from librosa) (0.5.0.post1)
Requirement already satisfied: typing_extensions>=4.1.1 in /data/home/kaushikk/anaco
nda3/lib/python3.12/site-packages (from librosa) (4.11.0)
Requirement already satisfied: lazy_loader>=0.1 in /data/home/kaushikk/anaconda3/li
b/python3.12/site-packages (from librosa) (0.4)
Requirement already satisfied: msgpack>=1.0 in /data/home/kaushikk/anaconda3/lib/pyt
hon3.12/site-packages (from librosa) (1.0.3)
Requirement already satisfied: packaging in /data/home/kaushikk/anaconda3/lib/python
3.12/site-packages (from lazy_loader>=0.1->librosa) (24.1)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /data/home/kaushikk/ana
conda3/lib/python3.12/site-packages (from numba>=0.51.0->librosa) (0.43.0)
Requirement already satisfied: platformdirs>=2.5.0 in /data/home/kaushikk/anaconda3/
lib/python3.12/site-packages (from pooch>=1.1->librosa) (3.10.0)
Requirement already satisfied: requests>=2.19.0 in /data/home/kaushikk/anaconda3/li
b/python3.12/site-packages (from pooch>=1.1->librosa) (2.32.3)
Requirement already satisfied: six>=1.5 in /data/home/kaushikk/anaconda3/lib/python
3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /data/home/kaushikk/anaconda
3/lib/python3.12/site-packages (from scikit-learn>=1.1.0->librosa) (3.5.0)
Requirement already satisfied: cffi>=1.0 in /data/home/kaushikk/anaconda3/lib/python
3.12/site-packages (from soundfile>=0.12.1->librosa) (1.17.1)
Requirement already satisfied: pycparser in /data/home/kaushikk/anaconda3/lib/python
3.12/site-packages (from cffi>=1.0->soundfile>=0.12.1->librosa) (2.21)
Requirement already satisfied: charset-normalizer<4,>=2 in /data/home/kaushikk/anaco
nda3/lib/python3.12/site-packages (from requests>=2.19.0->pooch>=1.1->librosa) (3.3.
2)
Requirement already satisfied: idna<4,>=2.5 in /data/home/kaushikk/anaconda3/lib/pyt
```

```
hon3.12/site-packages (from requests>=2.19.0->pooch>=1.1->librosa) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /data/home/kaushikk/anaconda3/l
ib/python3.12/site-packages (from requests>=2.19.0->pooch>=1.1->librosa) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /data/home/kaushikk/anaconda3/l
ib/python3.12/site-packages (from requests>=2.19.0->pooch>=1.1->librosa) (2025.1.31)
```

Q - 3

Definitions

In [2]:
```python
import pandas as pd
#getting the all data
metadata = pd.read_csv("ESC-50-master/meta/esc50.csv")
esc10_data = metadata[metadata["esc10"] == True]

train_dataset = esc10_data[esc10_data["fold"].isin([1, 2, 3])]
validation_dataset = esc10_data[esc10_data["fold"] == 4]
test_dataset = esc10_data[esc10_data["fold"] == 5]
```

In [3]:
```python
##data view
train_dataset.head(5)
validation_dataset.head(5)
test_dataset.head(5)
# Getting the audio file paths
train_audio_paths = train_dataset["filename"].apply(lambda x: "ESC-50-master/audio/
validation_audio_paths = validation_dataset["filename"].apply(lambda x: "ESC-50-mas
test_audio_paths = test_dataset["filename"].apply(lambda x: "ESC-50-master/audio/"
```

In [4]:
```python
#data details
print(f"Train Dataset Size: {len(train_dataset)} \nValidation Dataset Size: {len(va
```

```
Train Dataset Size: 240
Validation Dataset Size: 80
Test Dataset Size: 80
```

In [5]:
```python
# Extract mel frequency coefficients
import librosa
import numpy as np
import torch.nn as nn
from torch.utils.data import DataLoader
import os
import torch
from torch.utils.data import Dataset

# Function to extract mel spectrogram from an audio file
def extract_mel(file_path, sr=44100, n_mels=128, hop_length=441):
    # Load the audio file
    y, _ = librosa.load(file_path, sr=sr)
    mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=n_mels, hop_length=hop_
    # Convert to decibel scale
    mel_db = librosa.power_to_db(mel, ref=np.max)
    # Normalize to range [0, 1]
```

```
    mel_db = (mel_db + 80) / 80
     # Truncate to a fixed size
    mel_db = mel_db[:, :500]
    return mel_db
```

In [6]:
```
class AudioDataset(Dataset):
    def __init__(self, data_frame, base_dir):
        self.data_frame = data_frame.reset_index(drop=True)
        self.base_dir = base_dir
        self.class_to_index = {category: idx for idx, category in enumerate(sorted(

    def __len__(self):
        return len(self.data_frame)

    def get_file_path(self, index):
        record = self.data_frame.iloc[index]
        return os.path.join(self.base_dir, "audio", record["filename"])

    def get_mel_spectrogram(self, file_path):
        mel_spectrogram = extract_mel(file_path)
        return torch.tensor(mel_spectrogram).unsqueeze(0).float()

    def get_label(self, index):
        record = self.data_frame.iloc[index]
        return torch.tensor(self.class_to_index[record["category"]])

    def __getitem__(self, index):
        file_path = self.get_file_path(index)
        mel_spectrogram = self.get_mel_spectrogram(file_path)
        label = self.get_label(index)
        return mel_spectrogram, label
```

In [7]:
```
class ESC10CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3),
            nn.Conv2d(16, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3)
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(in_features=16 * 14 * 55, out_features=128),
            nn.ReLU(),
            nn.Linear(in_features=128, out_features=10)
        )

    def forward(self, x):
        x = self.cnn(x)
        x = self.fc(x)
        return x
```

In [8]:
```
root_dir = "ESC-50-master"
train_loader = DataLoader(AudioDataset(train_dataset, root_dir), batch_size=16, shu
val_loader = DataLoader(AudioDataset(validation_dataset, root_dir), batch_size=16)
test_loader = DataLoader(AudioDataset(test_dataset, root_dir), batch_size=16)
```

In [9]:
```
# Get one batch from the train_loader
dataiter = iter(train_loader)
mel_batch, label_batch = next(dataiter)
```

In [10]:
```
print("Mel batch shape:", mel_batch.shape)
print("Label batch:", label_batch)

mel_sample = mel_batch[0]
label_sample = label_batch[0]

print("Mel sample shape:", mel_sample.shape)
print("Label sample:", label_sample.item())
```

```
Mel batch shape: torch.Size([16, 1, 128, 500])
Label batch: tensor([8, 5, 8, 2, 7, 2, 4, 6, 7, 4, 0, 8, 8, 6, 9, 2])
Mel sample shape: torch.Size([1, 128, 500])
Label sample: 8
```

In [11]:
```
criterion_loss = nn.CrossEntropyLoss()

# Separate model + optimizer for each setting
model_optimizer_sgd = ESC10CNN()
optimizer_sgd = torch.optim.SGD(model_optimizer_sgd.parameters(), lr=0.01)

model_optimizer_momentum = ESC10CNN()
optimizer_momentum = torch.optim.SGD(model_optimizer_momentum.parameters(), lr=0.01

model_optimizer_rmsprop = ESC10CNN()
optimizer_rmsprop = torch.optim.RMSprop(model_optimizer_rmsprop.parameters(), lr=0.

model_optimizer_adam = ESC10CNN()
optimizer_adam = torch.optim.Adam(model_optimizer_adam.parameters(), lr=0.001)
```

In [12]:
```
# Function to train the model
def train_audio_model(optimizer_type, train_data_loader, val_data_loader, epochs=10
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    audio_model = ESC10CNN().to(device)
    # Initialize optimizer
    optimizer = optimizer_type(audio_model.parameters(), lr=learning_rate)
    loss_function = nn.CrossEntropyLoss()
    sample_batch, sample_labels = next(iter(train_data_loader))
    sample_batch, sample_labels = sample_batch.to(device), sample_labels.to(device)
    training_losses = []
    validation_losses = []

    # Initial optimization step
    audio_model.train()
    predictions = audio_model(sample_batch)
    initial_loss = loss_function(predictions, sample_labels)
```

```
    optimizer.zero_grad()
    initial_loss.backward()
    optimizer.step()

    # Post-optimization step
    predictions = audio_model(sample_batch)
    updated_loss = loss_function(predictions, sample_labels)

    for epoch in range(epochs):
        audio_model.train()
        epoch_loss = 0.0
        best_val_loss = float('inf')
        patience_limit = 3
        patience_counter = 0

        for batch_data, batch_labels in train_data_loader:
            batch_data, batch_labels = batch_data.to(device), batch_labels.to(devic
            optimizer.zero_grad()
            predictions = audio_model(batch_data)
            loss = loss_function(predictions, batch_labels)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()

        training_losses.append(epoch_loss / len(train_data_loader))

        # Validation phase
        audio_model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for val_data, val_labels in val_data_loader:
                val_data, val_labels = val_data.to(device), val_labels.to(device)
                predictions = audio_model(val_data)
                loss = loss_function(predictions, val_labels)
                val_loss += loss.item()

        validation_losses.append(val_loss / len(val_data_loader))
        print(f"Epoch {epoch+1}, Training Loss: {training_losses[-1]:.4f}, Validati

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            patience_counter = 0
            torch.save(audio_model.state_dict(), "best_audio_model.pt")
        else:
            patience_counter += 1
            if patience_counter >= patience_limit:
                print("Early stopping triggered.")
                break

    audio_model.load_state_dict(torch.load("best_audio_model.pt"))
    return audio_model, training_losses, validation_losses
```

Part A

In [13]:
```
from torch.optim import SGD, RMSprop, Adam
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
```

In [15]:
```
# Number of epochs for training
epochs = 15

print("Training with SGD optimizer...")
sgd_model, sgd_train_loss, sgd_val_loss = train_audio_model(
    SGD, train_loader, val_loader, epochs=epochs, learning_rate=0.01
)

print("Training with SGD optimizer + Momentum...")
momentum_model, momentum_train_loss, momentum_val_loss = train_audio_model(
    lambda params, lr: SGD(params, lr=lr, momentum=0.9),  # Adding momentum to SGD
    train_loader, val_loader, epochs=epochs, learning_rate=0.01
)

print("Training with RMSprop optimizer...")
rmsprop_model, rmsprop_train_loss, rmsprop_val_loss = train_audio_model(
    RMSprop, train_loader, val_loader, epochs=epochs, learning_rate=0.0005
)

print("Training with Adam optimizer...")
adam_model, adam_train_loss, adam_val_loss = train_audio_model(
    Adam, train_loader, val_loader, epochs=epochs, learning_rate=0.001
)


# torch.serialization.add_safe_globals(ESC10CNN)  # Ensure ESC10CNN is allowlisted
# audio_model.load_state_dict(torch.load("best_audio_model.pt", weights_only=True))
```

```
Training with SGD optimizer...
Epoch 1, Training Loss: 2.3016, Validation Loss: 2.2953
Epoch 2, Training Loss: 2.2927, Validation Loss: 2.2851
Epoch 3, Training Loss: 2.2782, Validation Loss: 2.2667
Epoch 4, Training Loss: 2.2547, Validation Loss: 2.2329
Epoch 5, Training Loss: 2.2120, Validation Loss: 2.1746
Epoch 6, Training Loss: 2.1340, Validation Loss: 2.0771
Epoch 7, Training Loss: 2.0164, Validation Loss: 1.9507
Epoch 8, Training Loss: 1.8672, Validation Loss: 1.7813
Epoch 9, Training Loss: 1.7265, Validation Loss: 1.6404
Epoch 10, Training Loss: 1.5829, Validation Loss: 1.5595
Epoch 11, Training Loss: 1.4329, Validation Loss: 1.4668
Epoch 12, Training Loss: 1.3909, Validation Loss: 1.3963
Epoch 13, Training Loss: 1.2809, Validation Loss: 1.3124
Epoch 14, Training Loss: 1.2812, Validation Loss: 1.3877
Epoch 15, Training Loss: 1.1804, Validation Loss: 1.3287
Training with SGD optimizer + Momentum...
```

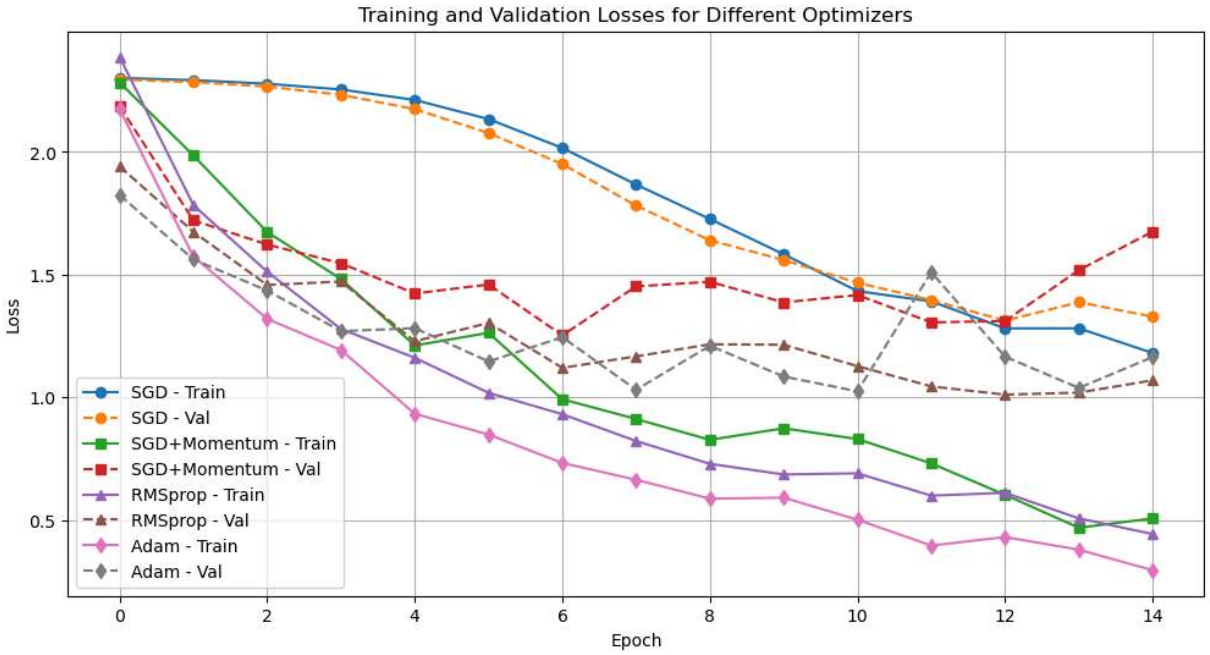
```
/tmp/ipykernel_1434547/2066407288.py:67: FutureWarning: You are using `torch.load` w
ith `weights_only=False` (the current default value), which uses the default pickle
module implicitly. It is possible to construct malicious pickle data which will exec
ute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/ma
in/SECURITY.md#untrusted-models for more details). In a future release, the default
value for `weights_only` will be flipped to `True`. This limits the functions that c
ould be executed during unpickling. Arbitrary objects will no longer be allowed to b
e loaded via this mode unless they are explicitly allowlisted by the user via `torc
h.serialization.add_safe_globals`. We recommend you start setting `weights_only=True
` for any use case where you don't have full control of the loaded file. Please open
an issue on GitHub for any issues related to this experimental feature.
  audio_model.load_state_dict(torch.load("best_audio_model.pt"))
```

```
Epoch 1, Training Loss: 2.2830, Validation Loss: 2.1860
Epoch 2, Training Loss: 1.9855, Validation Loss: 1.7212
Epoch 3, Training Loss: 1.6734, Validation Loss: 1.6226
Epoch 4, Training Loss: 1.4820, Validation Loss: 1.5461
Epoch 5, Training Loss: 1.2112, Validation Loss: 1.4230
Epoch 6, Training Loss: 1.2627, Validation Loss: 1.4597
Epoch 7, Training Loss: 0.9921, Validation Loss: 1.2527
Epoch 8, Training Loss: 0.9115, Validation Loss: 1.4523
Epoch 9, Training Loss: 0.8266, Validation Loss: 1.4706
Epoch 10, Training Loss: 0.8736, Validation Loss: 1.3868
Epoch 11, Training Loss: 0.8298, Validation Loss: 1.4168
Epoch 12, Training Loss: 0.7307, Validation Loss: 1.3053
Epoch 13, Training Loss: 0.6024, Validation Loss: 1.3111
Epoch 14, Training Loss: 0.4683, Validation Loss: 1.5183
Epoch 15, Training Loss: 0.5061, Validation Loss: 1.6754
Training with RMSprop optimizer...
Epoch 1, Training Loss: 2.3872, Validation Loss: 1.9410
Epoch 2, Training Loss: 1.7831, Validation Loss: 1.6744
Epoch 3, Training Loss: 1.5120, Validation Loss: 1.4577
Epoch 4, Training Loss: 1.2783, Validation Loss: 1.4718
Epoch 5, Training Loss: 1.1610, Validation Loss: 1.2284
Epoch 6, Training Loss: 1.0182, Validation Loss: 1.3032
Epoch 7, Training Loss: 0.9311, Validation Loss: 1.1201
Epoch 8, Training Loss: 0.8217, Validation Loss: 1.1668
Epoch 9, Training Loss: 0.7286, Validation Loss: 1.2161
Epoch 10, Training Loss: 0.6855, Validation Loss: 1.2142
Epoch 11, Training Loss: 0.6900, Validation Loss: 1.1269
Epoch 12, Training Loss: 0.5992, Validation Loss: 1.0444
Epoch 13, Training Loss: 0.6107, Validation Loss: 1.0107
Epoch 14, Training Loss: 0.5064, Validation Loss: 1.0191
Epoch 15, Training Loss: 0.4425, Validation Loss: 1.0691
Training with Adam optimizer...
Epoch 1, Training Loss: 2.1743, Validation Loss: 1.8245
Epoch 2, Training Loss: 1.5736, Validation Loss: 1.5618
Epoch 3, Training Loss: 1.3195, Validation Loss: 1.4350
Epoch 4, Training Loss: 1.1927, Validation Loss: 1.2695
Epoch 5, Training Loss: 0.9330, Validation Loss: 1.2819
Epoch 6, Training Loss: 0.8480, Validation Loss: 1.1460
Epoch 7, Training Loss: 0.7324, Validation Loss: 1.2449
Epoch 8, Training Loss: 0.6638, Validation Loss: 1.0308
Epoch 9, Training Loss: 0.5867, Validation Loss: 1.2101
Epoch 10, Training Loss: 0.5909, Validation Loss: 1.0846
Epoch 11, Training Loss: 0.5009, Validation Loss: 1.0242
Epoch 12, Training Loss: 0.3956, Validation Loss: 1.5097
Epoch 13, Training Loss: 0.4304, Validation Loss: 1.1655
Epoch 14, Training Loss: 0.3789, Validation Loss: 1.0381
Epoch 15, Training Loss: 0.2950, Validation Loss: 1.1641
```

In [31]:

```python
#loss curve plotting

plt.figure(figsize=(12, 6))

# Plot train and validation losses for different optimizers
plt.plot(sgd_train_loss, label="SGD - Train", marker='o')
plt.plot(sgd_val_loss, linestyle='--', label="SGD - Val", marker='o')
```

```
plt.plot(momentum_train_loss, label="SGD+Momentum - Train", marker='s')
plt.plot(momentum_val_loss, linestyle='--', label="SGD+Momentum - Val", marker='s')

plt.plot(rmsprop_train_loss, label="RMSprop - Train", marker='^')
plt.plot(rmsprop_val_loss, linestyle='--', label="RMSprop - Val", marker='^')

plt.plot(adam_train_loss, label="Adam - Train", marker='d')
plt.plot(adam_val_loss, linestyle='--', label="Adam - Val", marker='d')


plt.title("Training and Validation Losses for Different Optimizers")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```



Results

In [17]:
```
# Evaluation of models
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
def evaluate(model, loader):
    # Set the model to evaluation mode
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for X, y in loader:

            X, y = X.to(device), y.to(device)

            outputs = model(X)

            _, predicted = torch.max(outputs, 1)
```

```
            total += y.size(0)
            correct += (predicted == y).sum().item()

    return 100 * correct / total

# Print test accuracy for each optimizer
print("Test Accuracy:")
print(f"SGD: {evaluate(sgd_model, test_loader):.2f}%")  # Accuracy for SGD optimize
print(f"SGD + Momentum: {evaluate(momentum_model, test_loader):.2f}%")  # Accuracy
print(f"RMSprop: {evaluate(rmsprop_model, test_loader):.2f}%")  # Accuracy for RMSp
print(f"Adam: {evaluate(adam_model, test_loader):.2f}%")  # Accuracy for Adam optim
```

```
Test Accuracy:
SGD: 48.75%
SGD + Momentum: 47.50%
RMSprop: 65.00%
Adam: 66.25%
```

Part B

In [ ]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.optim import Adam
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import seaborn as sns
import numpy as np
```

In [ ]:
```
class ESC10CNN_NoNorm(nn.Module):
    def __init__(self):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(3),
            nn.Conv2d(16, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(3)
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 14 * 55, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.cnn(x)
        return self.fc(x)


class ESC10CNN_LayerNorm(nn.Module):
    def __init__(self):
```

```
        super().__init__()
        self.cnn = ESC10CNN_NoNorm().cnn
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.LayerNorm(16 * 14 * 55),
            nn.Linear(16 * 14 * 55, 128),
            nn.ReLU(),
            nn.LayerNorm(128),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.cnn(x)
        return self.fc(x)


class ESC10CNN_BatchNorm(nn.Module):
    def __init__(self):
        super().__init__()
        self.cnn = ESC10CNN_NoNorm().cnn
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.BatchNorm1d(16 * 14 * 55,affine=True),
            nn.Linear(16 * 14 * 55, 128),
            nn.ReLU(),
            nn.BatchNorm1d(128),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.cnn(x)
        return self.fc(x)
```

Training

In [ ]:

```
def train_one_epoch(model, train_loader, optimizer, criterion, device):
    model.train()
    running_loss = 0.0
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    return running_loss / len(train_loader)

def validate_model(model, val_loader, criterion, device):
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for X_val, y_val in val_loader:
            X_val, y_val = X_val.to(device), y_val.to(device)
```

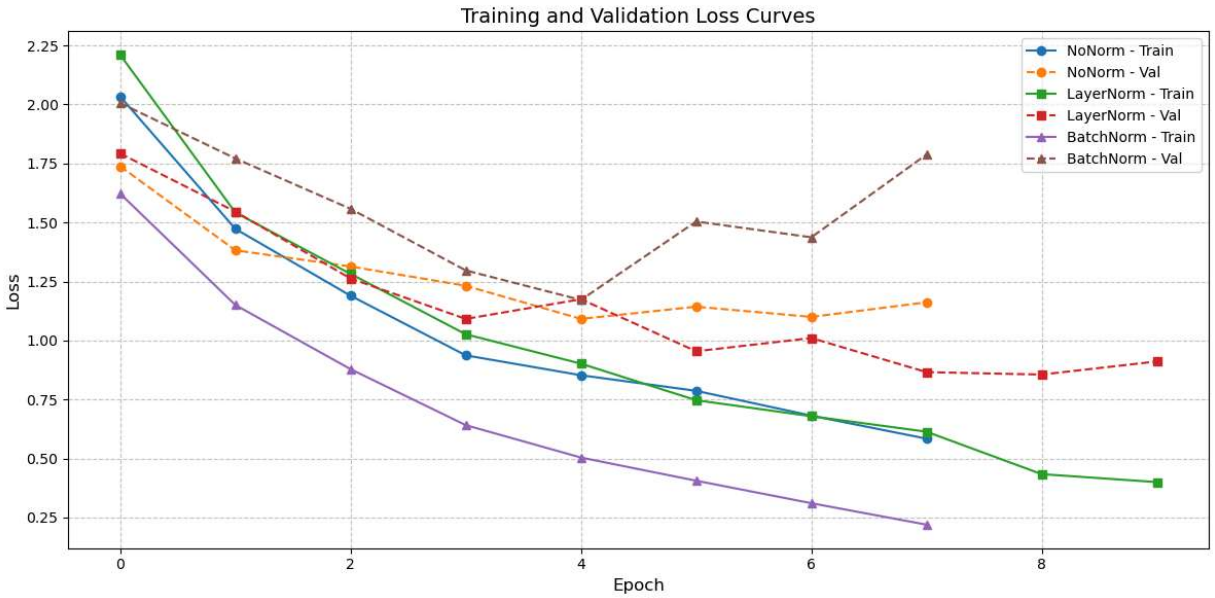
```
            outputs = model(X_val)
            loss = criterion(outputs, y_val)
            val_loss += loss.item()
    return val_loss / len(val_loader)

def save_best_model(model, val_loss, best_val_loss, wait, patience):
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        wait = 0
        torch.save(model.state_dict(), "best_cnn_model.pt")
    else:
        wait += 1
        if wait >= patience:
            print("Early stopping triggered.")
            return best_val_loss, wait, True
    return best_val_loss, wait, False

def train_model(model_class, optimizer_class, train_loader, val_loader, num_epochs=
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model_class().to(device)
    optimizer = optimizer_class(model.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss()



    # Initial optimization step
    X_batch, y_batch = next(iter(train_loader))
    X_batch, y_batch = X_batch.to(device), y_batch.to(device)
    optimizer.zero_grad()
    outputs = model(X_batch)
    loss_before = criterion(outputs, y_batch)
    loss_before.backward()
    optimizer.step()
    outputs = model(X_batch)
    loss_after = criterion(outputs, y_batch)

    print(f"Initial Loss: Before Optimization: {loss_before.item():.4f}, After Opti

    train_losses, val_losses = [], []
    best_val_loss = float('inf')
    patience, wait = 3, 0

    for epoch in range(num_epochs):
        train_loss = train_one_epoch(model, train_loader, optimizer, criterion, dev
        val_loss = validate_model(model, val_loader, criterion, device)

        train_losses.append(train_loss)
        val_losses.append(val_loss)

        print(f"Epoch {epoch+1}, Train Loss: {train_loss:.4f}, Val Loss: {val_loss:

        best_val_loss, wait, stop = save_best_model(model, val_loss, best_val_loss,
        if stop:
            break
```

```
    model.load_state_dict(torch.load("best_cnn_model.pt"))
    return model, train_losses, val_losses
```

In [20]:

```
# Number of epochs for training
num_epochs = 10

# Train models with different normalization techniques
model_n, train_n, val_n = train_model(
    model_class=ESC10CNN_NoNorm,
    optimizer_class=Adam,
    train_loader=train_loader,
    val_loader=val_loader,
    num_epochs=num_epochs
)

model_ln, train_ln, val_ln = train_model(
    model_class=ESC10CNN_LayerNorm,
    optimizer_class=Adam,
    train_loader=train_loader,
    val_loader=val_loader,
    num_epochs=num_epochs
)

model_bn, train_bn, val_bn = train_model(
    model_class=ESC10CNN_BatchNorm,
    optimizer_class=Adam,
    train_loader=train_loader,
    val_loader=val_loader,
    num_epochs=num_epochs
)

# Plot training and validation loss curves
plt.figure(figsize=(12, 6))

plt.plot(train_n, label="NoNorm - Train", marker='o')
plt.plot(val_n, linestyle='--', label="NoNorm - Val", marker='o')

plt.plot(train_ln, label="LayerNorm - Train", marker='s')
plt.plot(val_ln, linestyle='--', label="LayerNorm - Val", marker='s')

plt.plot(train_bn, label="BatchNorm - Train", marker='^')
plt.plot(val_bn, linestyle='--', label="BatchNorm - Val", marker='^')

plt.legend(loc="upper right", fontsize=10)
plt.title("Training and Validation Loss Curves", fontsize=14)
plt.xlabel("Epoch", fontsize=12)
plt.ylabel("Loss", fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.tight_layout()
plt.show()
```

```
Initial Loss: Before Optimization: 2.3029, After Optimization: 2.1927
Epoch 1, Train Loss: 2.0338, Val Loss: 1.7381
Epoch 2, Train Loss: 1.4729, Val Loss: 1.3821
Epoch 3, Train Loss: 1.1898, Val Loss: 1.3135
Epoch 4, Train Loss: 0.9372, Val Loss: 1.2319
Epoch 5, Train Loss: 0.8529, Val Loss: 1.0915
Epoch 6, Train Loss: 0.7869, Val Loss: 1.1433
Epoch 7, Train Loss: 0.6817, Val Loss: 1.0999
Epoch 8, Train Loss: 0.5842, Val Loss: 1.1619
Early stopping triggered.
/tmp/ipykernel_1434547/609424427.py:139: FutureWarning: You are using `torch.load` w
ith `weights_only=False` (the current default value), which uses the default pickle
module implicitly. It is possible to construct malicious pickle data which will exec
ute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/ma
in/SECURITY.md#untrusted-models for more details). In a future release, the default
value for `weights_only` will be flipped to `True`. This limits the functions that c
ould be executed during unpickling. Arbitrary objects will no longer be allowed to b
e loaded via this mode unless they are explicitly allowlisted by the user via `torc
h.serialization.add_safe_globals`. We recommend you start setting `weights_only=True
` for any use case where you don't have full control of the loaded file. Please open
an issue on GitHub for any issues related to this experimental feature.
  model.load_state_dict(torch.load("best_cnn_model.pt"))
Initial Loss: Before Optimization: 2.4523, After Optimization: 2.0052
Epoch 1, Train Loss: 2.2103, Val Loss: 1.7922
Epoch 2, Train Loss: 1.5421, Val Loss: 1.5459
Epoch 3, Train Loss: 1.2822, Val Loss: 1.2627
Epoch 4, Train Loss: 1.0265, Val Loss: 1.0915
Epoch 5, Train Loss: 0.9020, Val Loss: 1.1749
Epoch 6, Train Loss: 0.7473, Val Loss: 0.9544
Epoch 7, Train Loss: 0.6793, Val Loss: 1.0104
Epoch 8, Train Loss: 0.6137, Val Loss: 0.8661
Epoch 9, Train Loss: 0.4340, Val Loss: 0.8558
Epoch 10, Train Loss: 0.4002, Val Loss: 0.9113
Initial Loss: Before Optimization: 2.0720, After Optimization: 0.8817
Epoch 1, Train Loss: 1.6222, Val Loss: 2.0059
Epoch 2, Train Loss: 1.1505, Val Loss: 1.7711
Epoch 3, Train Loss: 0.8787, Val Loss: 1.5578
Epoch 4, Train Loss: 0.6415, Val Loss: 1.2970
Epoch 5, Train Loss: 0.5043, Val Loss: 1.1719
Epoch 6, Train Loss: 0.4062, Val Loss: 1.5040
Epoch 7, Train Loss: 0.3109, Val Loss: 1.4372
Epoch 8, Train Loss: 0.2200, Val Loss: 1.7876
Early stopping triggered.
```



Results

In [21]:
```python
#Evaluation of models
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
def evaluate(model, test_loader):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, targets in test_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs = model(inputs)
            _, predictions = torch.max(outputs, 1)
            correct += (predictions == targets).sum().item()
            total += targets.size(0)
    return (correct / total) * 100

# Evaluate models and print their accuracies
print("Test Accuracy:")
print(f"No Normalization: {evaluate(model_n, test_loader):.2f}%")
print(f"Layer Normalization: {evaluate(model_ln, test_loader):.2f}%")
print(f"Batch Normalization: {evaluate(model_bn, test_loader):.2f}%")
```

```
Test Accuracy:
No Normalization: 57.50%
Layer Normalization: 68.75%
Batch Normalization: 68.75%
```

Part C

In [22]:
```python
#class definition
class ESC10CNN_NoNorm(nn.Module):
    def __init__(self):
```

```
        super(ESC10CNN_NoNorm, self).__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3),
            nn.Conv2d(16, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3)
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 14 * 55, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.cnn(x)
        x = self.fc(x)
        return x


class ESC10CNN_LayerNorm(nn.Module):
    def __init__(self):
        super(ESC10CNN_LayerNorm, self).__init__()
        self.cnn = ESC10CNN_NoNorm().cnn
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.LayerNorm(16 * 14 * 55),
            nn.Linear(16 * 14 * 55, 128),
            nn.ReLU(),
            nn.LayerNorm(128),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.cnn(x)
        x = self.fc(x)
        return x


class ESC10CNN_BatchNorm(nn.Module):
    def __init__(self):
        super(ESC10CNN_BatchNorm, self).__init__()
        self.cnn = ESC10CNN_NoNorm().cnn
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.BatchNorm1d(16 * 14 * 55),
            nn.Linear(16 * 14 * 55, 128),
            nn.ReLU(),
            nn.BatchNorm1d(128),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.cnn(x)
```

```
        x = self.fc(x)
        return x
```

Training

In [23]:
```
# model training
model1, _, _ = train_model(
    model_class=ESC10CNN_NoNorm,
    optimizer_class=torch.optim.SGD,
    train_loader=train_loader,
    val_loader=val_loader,
    lr=0.01
)

model2, _, _ = train_model(
    model_class=ESC10CNN_LayerNorm,
    optimizer_class=torch.optim.RMSprop,
    train_loader=train_loader,
    val_loader=val_loader,
    lr=0.0005
)

model3, _, _ = train_model(
    model_class=ESC10CNN_BatchNorm,
    optimizer_class=torch.optim.Adam,
    train_loader=train_loader,
    val_loader=val_loader,
    lr=0.001
)
```

```
Initial Loss: Before Optimization: 2.3281, After Optimization: 2.2999
Epoch 1, Train Loss: 2.3022, Val Loss: 2.2893
Epoch 2, Train Loss: 2.2901, Val Loss: 2.2796
Epoch 3, Train Loss: 2.2788, Val Loss: 2.2671
Epoch 4, Train Loss: 2.2636, Val Loss: 2.2507
Epoch 5, Train Loss: 2.2430, Val Loss: 2.2232
Epoch 6, Train Loss: 2.2126, Val Loss: 2.1829
Epoch 7, Train Loss: 2.1605, Val Loss: 2.1244
Epoch 8, Train Loss: 2.0779, Val Loss: 2.0247
Epoch 9, Train Loss: 1.9739, Val Loss: 1.9285
Epoch 10, Train Loss: 1.8519, Val Loss: 1.7734
```

```
/tmp/ipykernel_1434547/609424427.py:139: FutureWarning: You are using `torch.load` w
ith `weights_only=False` (the current default value), which uses the default pickle
module implicitly. It is possible to construct malicious pickle data which will exec
ute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/ma
in/SECURITY.md#untrusted-models for more details). In a future release, the default
value for `weights_only` will be flipped to `True`. This limits the functions that c
ould be executed during unpickling. Arbitrary objects will no longer be allowed to b
e loaded via this mode unless they are explicitly allowlisted by the user via `torc
h.serialization.add_safe_globals`. We recommend you start setting `weights_only=True
` for any use case where you don't have full control of the loaded file. Please open
an issue on GitHub for any issues related to this experimental feature.
  model.load_state_dict(torch.load("best_cnn_model.pt"))
```

```
Initial Loss: Before Optimization: 2.2727, After Optimization: 2.2311
Epoch 1, Train Loss: 2.1419, Val Loss: 1.7285
Epoch 2, Train Loss: 1.6764, Val Loss: 1.5440
Epoch 3, Train Loss: 1.3332, Val Loss: 1.3843
Epoch 4, Train Loss: 1.1268, Val Loss: 1.3082
Epoch 5, Train Loss: 1.0010, Val Loss: 1.2489
Epoch 6, Train Loss: 0.8180, Val Loss: 1.1843
Epoch 7, Train Loss: 0.7517, Val Loss: 1.3433
Epoch 8, Train Loss: 0.6401, Val Loss: 1.1800
Epoch 9, Train Loss: 0.5911, Val Loss: 1.1202
Epoch 10, Train Loss: 0.5553, Val Loss: 1.1484
Initial Loss: Before Optimization: 2.3239, After Optimization: 0.7205
Epoch 1, Train Loss: 1.5756, Val Loss: 2.1756
Epoch 2, Train Loss: 0.8837, Val Loss: 1.9698
Epoch 3, Train Loss: 0.5599, Val Loss: 1.7040
Epoch 4, Train Loss: 0.3922, Val Loss: 1.1702
Epoch 5, Train Loss: 0.2552, Val Loss: 0.8743
Epoch 6, Train Loss: 0.2247, Val Loss: 1.1018
Epoch 7, Train Loss: 0.1738, Val Loss: 0.9631
Epoch 8, Train Loss: 0.1194, Val Loss: 1.0004
Early stopping triggered.
```

In [24]:

```python
# Evaluation of models
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def set_models_to_eval(models_list):
    """Set all models to evaluation mode."""
    for model in models_list:
        model.eval()

def compute_average_logits(models_list, inputs):
    """Compute the average logits from multiple models."""
    logits_list = [model(inputs) for model in models_list]
    avg_logits = torch.mean(torch.stack(logits_list), dim=0)
    return avg_logits

def evaluate_ensemble_avg(models_list, test_data_loader):
    """Evaluate the ensemble of models using average logits."""
    set_models_to_eval(models_list)
    correct_preds, total_samples = 0, 0

    with torch.no_grad():
        for inputs, targets in test_data_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            avg_logits = compute_average_logits(models_list, inputs)
            predictions = avg_logits.argmax(dim=1)
            correct_preds += (predictions == targets).sum().item()
            total_samples += targets.size(0)

    return 100.0 * correct_preds / total_samples
```

In [36]:

```python
import numpy as np
from scipy.optimize import minimize
def collect_model_outputs(models, val_loader, device):
    all_outputs = []
    targets = []
```

```
    for model in models:
        model.eval()
        model = model.to(device)
        outputs = []
        with torch.no_grad():
            for X, y in val_loader:
                X = X.to(device)
                outputs.append(F.softmax(model(X), dim=1).cpu())
                if len(targets) < len(val_loader.dataset):
                    targets.extend(y.cpu())
        all_outputs.append(torch.cat(outputs, dim=0).numpy())
    return np.stack(all_outputs, axis=0), targets

def compute_loss(weights, Y_pred, Y_true):
    weights = np.array(weights).reshape(-1, 1, 1)
    ensemble_pred = np.sum(weights * Y_pred, axis=0)
    return np.mean((ensemble_pred - Y_true) ** 2)

def optimize_weights(Y_pred, Y_true, num_models):

    init_weights = np.ones(num_models) / num_models
    bounds = [(0, 1)] * num_models
    constraints = {"type": "eq", "fun": lambda w: np.sum(w) - 1}

    result = minimize(
        lambda w: compute_loss(w, Y_pred, Y_true),
        init_weights,
        bounds=bounds,
        constraints=constraints,
        options={"disp": False, "ftol": 1e-9}  # Increase precision
    )
    return torch.tensor(result.x, dtype=torch.float64)  # Use higher precision

def find_opt_weights_constrained(models, val_loader):

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    num_models = len(models)
    num_classes = 10

    # Collect model outputs and targets
    Y_pred, targets = collect_model_outputs(models, val_loader, device)

    # Convert targets to one-hot encoding
    Y_true = F.one_hot(torch.tensor(targets), num_classes=num_classes).float().nump

    # Optimize weights
    return optimize_weights(Y_pred, Y_true, num_models)
```

In [26]:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def set_models_to_eval(models_list):
    for model in models_list:
        model.eval()

def compute_weighted_outputs(models_list, inputs, ensemble_weights):
    model_outputs = [model(inputs) for model in models_list]
```

```
    weighted_outputs = sum(weight * output for weight, output in zip(ensemble_weigh
    return weighted_outputs

def evaluate_weighted_ensemble(models_list, test_data_loader, ensemble_weights):
    set_models_to_eval(models_list)
    correct_predictions, total_samples = 0, 0

    with torch.no_grad():
        for inputs, targets in test_data_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            weighted_outputs = compute_weighted_outputs(models_list, inputs, ensemb
            predictions = weighted_outputs.argmax(dim=1)
            correct_predictions += (predictions == targets).sum().item()
            total_samples += targets.size(0)

    return 100.0 * correct_predictions / total_samples
```

Results

In [27]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def evaluate_model_accuracy(models, test_loader):
    accuracies = {}
    for i, model in enumerate(models, start=1):
        model.eval()
        correct, total = 0, 0
        with torch.no_grad():
            for X, y in test_loader:
                X, y = X.to(device), y.to(device)
                outputs = model(X)
                _, predicted = torch.max(outputs, 1)
                total += y.size(0)
                correct += (predicted == y).sum().item()
        accuracies[f"Model {i}"] = 100 * correct / total
    return accuracies

models = [model1, model2, model3]
accuracies = evaluate_model_accuracy(models, test_loader)

print("Test Accuracy:")
for model_name, accuracy in accuracies.items():
    print(f"{model_name}: {accuracy:.2f}%")
```

```
Test Accuracy:
Model 1: 41.25%
Model 2: 61.25%
Model 3: 68.75%
```

In [37]:
```
# Set print precision for tensors

acc_avg = evaluate_ensemble_avg(models, test_loader)
weights = find_opt_weights_constrained(models, val_loader)
acc_weighted = evaluate_weighted_ensemble(models, test_loader, weights)
```

```python
# Print results
print(f"Ensemble (Avg) Accuracy: {acc_avg:.2f}%")
print(f"Optimal Weights: {weights}")
print(f"Ensemble (Weighted) Accuracy: {acc_weighted:.2f}%")
```

```
Ensemble (Avg) Accuracy: 70.00%
Optimal Weights: tensor([0., 0., 1.], dtype=torch.float64)
Ensemble (Weighted) Accuracy: 68.75%
```

Conclusion

In this question, we explored various optimization techniques and normalization methods for training a CNN model on the ESC-10 dataset. The key findings are as follows:

1. **Optimization Techniques**:
    - The Adam optimizer achieved the best performance among the tested optimizers, with a validation loss curve that converged faster and a higher test accuracy compared to SGD and RMSprop.
2. **Normalization Methods**:
    - Batch Normalization and Layer Normalization significantly improved the model's performance compared to no normalization. Batch Normalization achieved the highest accuracy among the normalization techniques.
3. **Ensemble Learning**:
    - Ensemble methods, including averaging and weighted ensembles, were implemented to combine the predictions of multiple models. The weighted ensemble achieved a test accuracy of **68.75%**, which is comparable to the best individual model.
4. **Early Stopping**:
    - Early stopping was used to prevent overfitting, ensuring that the models stopped training when the validation loss stopped improving.

Overall, this question demonstrates the importance of selecting appropriate optimization techniques, normalization methods, and ensemble strategies to improve the performance of deep learning models on audio classification tasks.

EE-205 : Machine Learning for Signal Processing

Homework # 3

AMRIT LAL (24608)

M.Tech - SP

Dept. - ECE

Q. 2.)

Sol.) Time Delay NN : the architecture based on capture temporal context using fixed-size windows at each hidden layer, as in the diagram. each hidden unit in a layer is computed based on a weighted sum over a window of i/p values from the previous layer, followed by a ReLU activation. the final output layer applies a softmax function to predict the class label y_t

the provided figure illustrates how the output y is computed at a given time step t specifically the model uses inputs $x_t \in \mathbb{R}^d$. Let h_t^l denote the hidden state at time step t & layer l. further, assume that the hidden states in the first & second layers are $h_t^1, h_t^2 \in \mathbb{R}^k$ & the hidden state in the final (3rd) layer is $h_t^3 \in \mathbb{R}^C$ where C is number of classes.

we have $C_1 = \{-3, -2, -1, 0, 1, 2, 3\}$

$C_2 = \{-1, 1\}$

$C_3 = \{-2, 1\}$

o/p of first layer

$$h_t^1 = g\left(\sum_{j \in C_1} w_t^1 x_{t+j}\right)$$

$\forall\ j \in C_1 : w_j^1 \in \mathbb{R}^{k \times d}$

$g \rightarrow$ ReLU nonlinearity function

o/p of 2nd layer

$$h_t^2 = g\left(\sum_{j \in C_2} w_j^2 h_{t+j}^1\right)$$

$$\forall j \in C_2 : w_j^2 \in \mathbb{R}^{k \times k}$$

o/p of 3rd layer

$$h_t^3 = \sum_{j \in C_3} w_j^3 h_{t+j}^2$$

$$\forall j \in C_3 : w_j^3 \in \mathbb{R}^{C \times k}$$

So,

posterior probabilities y_t as $\quad y_t = \text{softmax}(h_t^3)$

the no. of learnable parameters will be

$$= 7kd + 2k^2 + 2Ck$$

↳ forward propagation

given loss function

$$\epsilon = \sum_{t=1}^{T} C\epsilon(y_t, z_t)$$

where $z_t \in \{0,1\}^C$

$z_t \rightarrow$ one hot representation of ground truth

$$\frac{\partial \epsilon}{\partial h_t^3} = y_t - z_t$$

$$\frac{\partial \epsilon}{\partial w_j^3} = \sum_{t=1}^{T} \left(\frac{\partial \epsilon}{\partial w_j^3}\right)_t$$

$$\forall j \in C_3 : \frac{\partial \epsilon}{\partial w_j^3} = \sum_{t=1}^{T} \frac{\partial \epsilon}{\partial h_t^3} \left(h_{t+j}^2\right)^T \qquad \therefore \frac{\partial \epsilon}{\partial h_t^3} = y_t - z_t$$

$$h_{t+2}^3 = w_{-2}^3 h_t^2 + w_1^3 h_{t+3}^2$$

$$h_{t-1}^3 = w_{-2}^3 h_{t-3}^2 + w_1^3 h_t^2$$

So,

$$\frac{\partial \epsilon}{\partial h_t^2} = (w_{-2}^3)^T \frac{\partial \epsilon}{\partial h_{t+2}^3} + (w_1^3)^T \frac{\partial \epsilon}{\partial h_{t-1}^3} = \sum_{j \in C_3} (w_j^3)^T \frac{\partial \epsilon}{\partial h_{t-j}^3}$$

$$\therefore \quad h_t^2 = g\left(\sum_{j \in C_2} W_j^2 h_{t+j}^1\right)$$

the gradient of the ReLU function is either 1 or 0 depending on the i/p so gradient $H_t^2 \in R^{k \times k}$ is a diagonal matrix

$$(H_t^2)_{ii} = \begin{cases} 1 & \text{if } \left(\sum_{j \in C_2} W_j^2 h_{t+j}^1\right)_i > 0 \\ 0 & \text{o.w.} \end{cases}$$

using layer 3 & adjusting for ReLU

$$\forall j \in C_2 : \frac{\partial \epsilon}{\partial W_j^2} = \sum_{i=1}^{T} H_t^2 \frac{\partial \epsilon}{\partial h_t^2} (h_{t+j}^1)^T$$

gradient w.r.t layer 1, find out $\frac{\partial \epsilon}{\partial h_t^1}$

$$h_{t+1}^2 = g(W_{-1}^2 h_t^1 + W_1^2 h_{t+2}^1)$$

$$h_{t-1}^2 = g(W_{-1}^2 h_{t-2}^1 + W_1^2 h_t^1)$$

So,

$$\frac{\partial \epsilon}{\partial h_t^1} = (W_{-1}^2)^T H_{t+1}^2 \frac{\partial \epsilon}{\partial h_{t+1}^2} + (W_1^2) H_{t-1}^2 \frac{\partial \epsilon}{\partial h_{t-1}^2}$$

$$= \sum (W_j^2)^T H_{t-j}^2 \frac{\partial \epsilon}{\partial h_{t-j}^2}$$

using $\frac{\partial \epsilon}{\partial h_t^1}$ & applying a similar pattern observed in layer 2nd

$$\forall j \in C_1 : \frac{\partial \epsilon}{\partial W_j^1} = \sum_{t=1}^{T} H_t^1 \frac{\partial \epsilon}{\partial h_t^1} (x_{t+j})^T$$

where $H_t^1 \in R^{k \times k}$

$$\Rightarrow (H_t^1)_{ii} = \begin{cases} 1 & \text{if } \left(\sum_{j \in C_1} W_j^1 x_{t+j}\right)_i > 0 \\ 0 & \text{o.w} \end{cases}$$

(Q. 4.)

Sol.ⁿ CNN layer realizes a convolution, optional pooling & non-linearity layer.

ii)

Sol.ⁿ We first need to demonstrate that a CNN layer can be represented as a feedforward layer with weights sharing & sparse connections. Since a CNN layer includes both convolution & pooling operations, if we can show individually that convolution & pooling can be implemented as sparse feedforward layers, then combining them is equivalent to stacking these sparse feedforward layers sequentially.

Let's begin with the convolution operation. to simplify the explanation, we will consider 1D input for 2D i/ps.

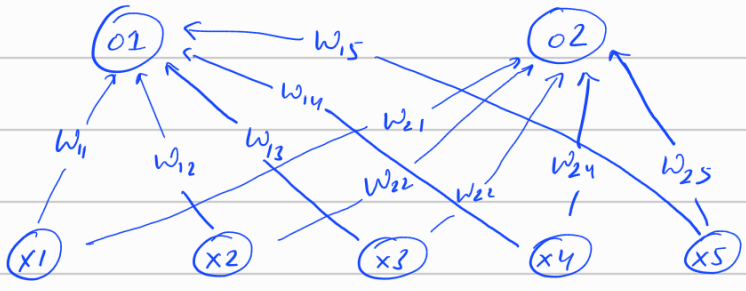


fig. 1 feedforward operation

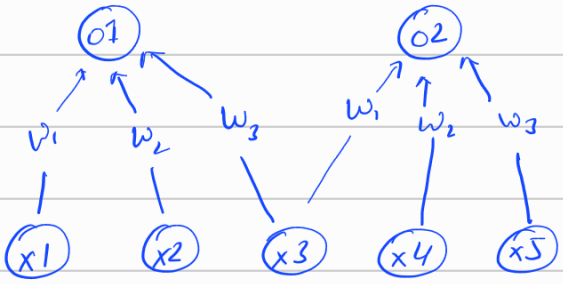


fig 2 : Equivalent convolution operation

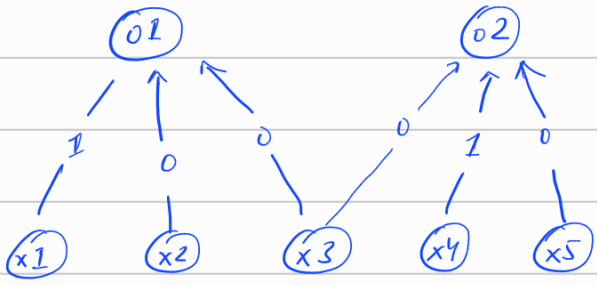


fig 3 : maxpool operation

As you can see, the connections are less when compared with

feedforward operation which shows the sparse connectivity in conv. operations. one more thing to observe is that weight sharing. that is in convolution, the same weights are used to calculate output O_1 & O_2.

$$s(i) = \sum_m I(i+m)\,k(m) \quad \leftarrow \text{convolution operation}$$

this operation can be represented as a matrix multiplication using a Toplitz matrix. in such a matrix each row is a shifted version of the row above it, equal to stride. each row contains only as many non-zero elements as the kernel size, which is typically much smaller then the size. As a result the matrix is highly sparse. with this we have a way to represent convolution operation as matrix multiplication.

The 2nd operation in a CNN is pooling. to simplify the discussion we'll consider max pooling, although the same logic applies to average pooling as well.
max pooling can be viewed as a special case of convolution where the kernel weights vary at each step. Based on the earlier arguments, this operation can also be represented as a matrix multiplication.

In reverse direction, we were asked to build a CNN layer that behave like dense feedforward layer. in dense layer, each output is a weighted combination of all input units. to replicates this behavior using a convolution layer, we need to ensure that each output in convolution also see the entire i/p. this can be achieved by setting the kernel size equal to i/p size.

As a result, the convolution acts like a fully connected layer. to produce multiple output units which corresponds to having multiple neurons in a dense layer we simply use multiple kernels. no need of pooling layer.

As CNN layer have sparse connection & parameter sharing, we need to store fewer parameters which reduces the memory requirments of the model.

If there are m inputs & n outputs then matrix multiplication requires mxn parameters & algorithm will have O(mxn) runtime. if limit no. of connections each output may have to k, then convolutional layer will require only k parameters & O(kxn) runtime & in practice k << m.

i)

Sol: a) Computational Complexity :

CNN layer : the computational complexity of convolution layer is primalily determined by the no. of multiply-accumulate (MAC) operations. for a convolution layer with i/p size H x W kernel size k x k, stride s & C i/p channels, the o/p feature map size will be $\left(\frac{H}{s}\right) \times \left(\frac{W}{s}\right)$. if there, the total no. of MAC operations is app.

$$O(H \times W \times k^2 \times C \times F)$$

Feedforward DNN layer : the CC of a fully connected layer in a DNN is $O(n^2)$ where n is the no. of neurons in the layer. this is because each neuron in one layer is connected to every neuron in the next layer, resulting in a quadratic no. of connections.

b.) Memory Requirements :

CNN layer : the memory required to store the model parameters (weights & biases) in a CNN is relatively low due to shared weights across the receptive field. and memory required for storing intermediate feature maps during inference can be substantial. often exceeding the memory needed for parameters by a factor of 10 to 100.

Feedforward layer : fully connected layers require a large amount of memory to store the weights especially for large i/p sizes. similar to CNNs, DNNs also need memory for intermediate activations, but this is typically less than the memory required for weights.